

Ortus — Guia das Mudanças

Data: 28 de julho de 2026
Commits: `de5fc7a` (Grandes mudanças) · `3b51f79` (Reorganiza Supabase)
Repositório: `github.com/JulianoFVS/Dev`

1. Resumo do que foi adicionado hoje

Hoje o Ortus recebeu duas grandes entregas:

Área	O que mudou
Comunicação	WhatsApp, e-mail e SMS integrados na agenda e ficha do paciente
Lembretes automáticos	Cron diário envia lembretes de consulta
Supabase Fase 1	Lembretes, horários de profissionais e taxas de recebimento em tabelas SQL
Supabase Fase 2	Prontuário relacional (tratamentos, anamneses, documentos, evoluções)
Ferramentas	Scripts de migration, audit e documentação em <code>supabase/README.md</code>

2. Onde encontrar cada coisa

2.1 Comunicação com pacientes

Item	Caminho
Botões WhatsApp / E-mail / SMS	<code>components/PatientContactButtons.tsx</code>
Lógica de templates e envio	<code>lib/comunicacao.ts</code>
SMS (Twilio)	<code>lib/sms.ts</code>
E-mail (Resend)	<code>lib/email.ts</code>
Templates padrão	<code>lib/configDefaults.ts</code>
Status da integração (API)	<code>app/api/comunicacao/status/route.ts</code>

Onde aparece na interface:

- Agenda (`/agenda`)
- Lista de pacientes (`/pacientes`)
- Ficha do paciente (`/pacientes/[id]`)
- Modal de ações rápidas (`PatientActionModal`)
- Painel lateral do paciente (`PatientSlideOver`)

2.2 Lembretes automáticos de agenda

Item	Caminho
Processamento de lembretes	<code>lib/lembretesAgenda.ts</code>

API cron (Vercel)	app/api/lembretes/processar/route.ts
Agendamento do cron	vercel.json (todos os dias às 10h BRT)
Tabela no banco	lembretes_agenda

2.3 Reorganização Supabase

Item	Caminho
Documentação	supabase/README.md
Migrations	supabase/migrations/
Chaves de config tipadas	lib/configKeys.ts
Client admin (server)	lib/supabaseAdmin.ts
Prontuário — repositórios	lib/db/
Carregar prontuário completo	lib/fichaPaciente.ts

2.4 Scripts de banco

Script	Comando	Função
Aplicar migration	npm run db:migrate OU node scripts/apply-migration.mjs NOME.sql	Executa SQL no Supabase
Auditar schema	npm run db:audit	Lista tabelas e colunas
Gerar tipos TS	npm run db:types	Cria lib/database.types.ts
Testar conexão	node scripts/test-connection.mjs	Valida DATABASE_URL

3. Configuração — o que fazer primeiro

3.1 Arquivo .env.local

Copie de .env.example e preencha:

```
# Supabase (obrigatório)
NEXT_PUBLIC_SUPABASE_URL=https://SEU_REF.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=...
SUPABASE_SERVICE_ROLE_KEY=...
SUPABASE_PROJECT_REF=SEU_REF

# Banco — use o POOLER (porta 6543), não a conexão direct
DATABASE_URL=postgresql://postgres.SEU_REF:SENHA@aws-1-sa-east-1.pooler.supabase.com:6543/postgres

# SMS (opcional — Twilio)
```

```
TWILIO_ACCOUNT_SID=
TWILIO_AUTH_TOKEN=
TWILIO_FROM_NUMBER=

# E-mail (opcional - Resend)
RESEND_API_KEY=
RESEND_FROM_EMAIL=Ortus <contato@sua-clinica.com.br>
```

Importante: A conexão Direct (`db.xxx.supabase.co:5432`) pode falhar em redes sem IPv6. Sempre prefira o `pooler`.

3.2 Migrations já aplicadas (produção)

As migrations abaixo **já foram aplicadas** no projeto Supabase `vjqeeekvoxddxwvbazwlt` :

1. `20260729 lembretes_horarios_recebimento.sql` — Fase 1
2. `20260730_ficha_relacional.sql` — Fase 2

Só rode de novo se estiver configurando **outro ambiente** (staging, clone):

```
node scripts/apply-migration.mjs 20260729_lembretes_horarios_recebimento.sql
node scripts/apply-migration.mjs 20260730_ficha_relacional.sql
```

3.3 Rodar o projeto

```
npm install
npm run dev
```

Validar build antes de deploy:

```
npm run build
```

4. Novas tabelas no Supabase

Fase 1 — Operação e financeiro

Tabela	Para quê	Substitui
<code>lembretes_agenda</code>	Registra lembretes já enviados por canal	JSON <code>lembretes_enviados</code> em config
<code>profissionais_horarios</code>	Horário de cada profissional por clínica	JSON <code>horario_profissional_*</code>
<code>ortus_schema_migrations</code>	Controle de migrations locais	—
Colunas em <code>agendamentos</code>	<code>valor_bruto</code> , <code>valor_liquido</code> , <code>taxa_id</code> , <code>taxa_nome</code> , <code>taxa_percentual</code>	JSON <code>lancamentos_meta</code>

Fase 2 — Prontuário relacional

Tabela	Conteúdo
paciente_tratamentos	Procedimentos, dente, valor, status, data
paciente_anamneses	Respostas por modelo, snapshot das perguntas
paciente_documentos	Nome, tipo, caminho no Storage, metadados
paciente_evolucoes	Texto clínico, data, profissional

O que ficou no JSON `ficha_medica` :

- Odontograma
- Mapa facial (HOF) e fotos
- Texto livre do odontograma
- Outros campos gráficos legados

Os dados antigos do JSON foram **migrados automaticamente** para as tabelas na Fase 2.

5. Como usar cada funcionalidade nova

5.1 Comunicação (WhatsApp, E-mail, SMS)

Configurar templates:

1. Acesse **Configurações** (`/configuracoes`)
2. Aba de comunicação / templates
3. Edite mensagens para eventos: lembrete, pós-consulta, orçamento etc.

Enviar manualmente:

1. Abra um paciente na agenda ou na ficha
2. Clique nos ícones verde (WhatsApp), azul (e-mail) ou roxo (SMS)
3. WhatsApp abre com mensagem pré-preenchida; e-mail/SMS usam Resend/Twilio se configurados

Verificar se SMS/e-mail estão ativos:

- GET `/api/comunicacao/status` (ou veja logs ao enviar)

5.2 Lembretes automáticos

Como funciona:

- Todo dia às **10h (horário de Brasília)** a Vercel chama `/api/lembretes/processar?enviar=true`
- Busca consultas do dia seguinte
- Envia lembrete (conforme templates) e grava em `lembretes_agenda`

Testar manualmente (dev):

```
GET http://localhost:3000/api/lembretes/processar?enviar=false
```

(com `enviar=false` só lista, não envia)

Na agenda:

- Ao marcar lembrete como enviado manualmente, grava canal em `lembretes_agenda`

5.3 Horários de profissionais

Onde configurar:

- **Ajustes** → **Equipe** (`/ajustes/equipe`)
- Edite horário, intervalo, dias da semana e limite simultâneo

Onde fica salvo:

- Tabela `profissionais_horarios`
- Fallback legado em `configuracoes_clinica` se a tabela não existir

5.4 Recebimento com taxa de maquininha

Onde usar:

- Agenda — ao receber pagamento de consulta
- Painel lateral do paciente (`PatientSlideOver`)

O que é gravado:

- Valor bruto, líquido e dados da taxa nas colunas de `agendamentos`

5.5 Modelos de anamnese

Onde configurar:

- **Configurações** (`/configuracoes`) — criar/editar modelos

Onde ficam salvos:

- Supabase: chave `anamnese_modelos` em `configuracoes_clinica`
- Fallback: `localStorage`

5.6 Prontuário do paciente (Fase 2)

Onde usar: `/pacientes/[id]`

Aba	O que faz	Onde salva
Tratamentos	CRUD de procedimentos	<code>paciente_tratamentos</code>
Anamnese	Preencher e salvar fichas	<code>paciente_anamneses</code>
Documentos	Upload de arquivos	Storage + <code>paciente_documentos</code>
Evolução	Registros clínicos	<code>paciente_evolucoes</code>
Odontograma / HOF	Mapas gráficos	JSON <code>ficha_medica</code>

Teste rápido pós-deploy:

1. Crie um tratamento → recarregue a página → deve persistir
2. Repita para anamnese, documento e evolução

Formulário rápido de tratamento:

- `components/forms/TreatmentForm.tsx` (modais de ação rápida)

5.7 Lista e exportação de pacientes

Melhorias:

- Filtro por procedimento pendente usa `paciente_tratamentos`
- Export CSV inclui totais de anamneses, tratamentos e documentos das tabelas SQL

6. Camada de código `lib/db/`

Repositórios criados para acesso ao prontuário:

Arquivo	Funções principais
<code>lib/db/tratamentos.ts</code>	<code>listarTratamentos</code> , <code>criarTratamento</code> , <code>atualizarTratamento</code> , <code>excluirTratamento</code>
<code>lib/db/anamneses.ts</code>	<code>listarAnamneses</code> , <code>criarAnamnese</code> , <code>excluirAnamnese</code>
<code>lib/db/documentos.ts</code>	<code>listarDocumentos</code> , <code>criarDocumento</code> , <code>excluirDocumento</code>
<code>lib/db/evolucoes.ts</code>	<code>listarEvolucoes</code> , <code>criarEvolucao</code> , <code>excluirEvolucao</code>
<code>lib/db/fichaClinica.ts</code>	<code>carregarFichaClinica</code> , <code>salvarFichaClinica</code> (odontograma/HOF)
<code>lib/fichaPaciente.ts</code>	<code>carregarProntuario</code> — une SQL + JSON com fallback legado

Importar tudo:

```
import { criarTratamento, listarTratamentos } from '@lib/db/tratamentos';
import { carregarProntuario } from '@lib/fichaPaciente';
```

7. Deploy na Vercel

1. Conecte o repositório `JulianoFVS/Dev`
2. Configure as variáveis de ambiente (mesmas do `.env.local`)
3. O cron em `vercel.json` roda automaticamente em produção
4. Para SMS/e-mail automáticos nos lembretes, configure Twilio e Resend na Vercel

Cron configurado:

- Horário UTC: `0 13 * * *` (= 10h BRT)
- Rota: `/api/lembretes/processar?enviar=true`

8. Checklist pós-atualização

- ☐ `.env.local` com pooler (6543) e chaves Supabase
- ☐ `npm run dev` — app sobe sem erro
- ☐ Abrir paciente — tratamentos/anamneses carregam
- ☐ Criar tratamento e recarregar — persiste
- ☐ Horários em Ajustes → Equipe salvam
- ☐ (Opcional) Twilio/Resend configurados para SMS/e-mail
- ☐ (Opcional) `npm run db:types` para tipos TypeScript

- ☐ Deploy Vercel com variáveis de ambiente

9. Legado e compatibilidade

O código mantém **fallback** para dados antigos:

Recurso novo	Fallback legado
lembretes_agenda	configuracoes_clinica.lembretes_enviados
profissionais_horarios	JSON horario_profissional_*
Taxas em agendamentos	lancamentos_meta
Tabelas do prontuário	Arrays dentro de ficha_medica JSON
Modelos anamnese Supabase	localStorage

Isso garante que clínicas com dados antigos continuem funcionando até a migration rodar.

10. Suporte e referências

Documento	Local
README Supabase	supabase/README.md
Migrations SQL	supabase/migrations/
Variáveis de ambiente	.env.example
Chaves de config	lib/configKeys.ts

Projeto Supabase: região São Paulo · ref `vjqeeekvoxddxwvbazw1t`